Support Vector Machine

SVM

Dr. Mauricio Toledo-Acosta
mauricio.toledo@unison.mx

Table of Contents

1. Introducción
2. Support Vector Machine
   - SVM de margen suave
3. Kernel Trick

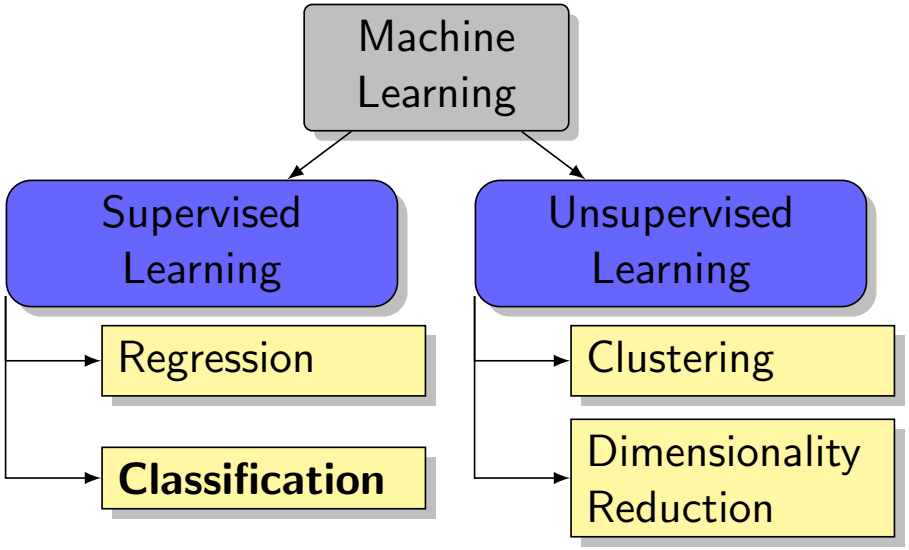

Machine Learning
Supervised Learning
Unsupervised Learning
Regression
Classification
Clustering
Dimensionality Reduction

Table of Contents

1 Introducción

2 Support Vector Machine
- SVM de margen suave

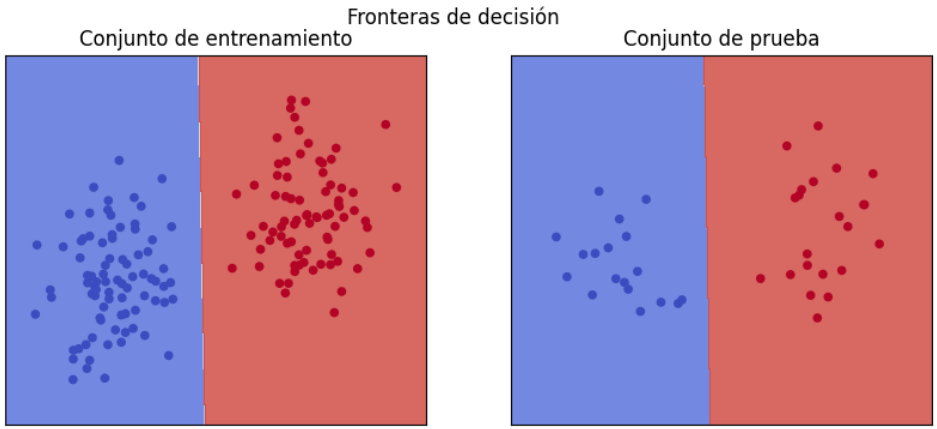
3 Kernel Trick

Introducción

Support Vector Machine

Modelo supervisado de clasificación binaria que busca encontrar una frontera de decisión óptima que separe las clases de puntos.

Su principal objetivo es encontrar el hiperplano que mejor separa las clases en un espacio de características.

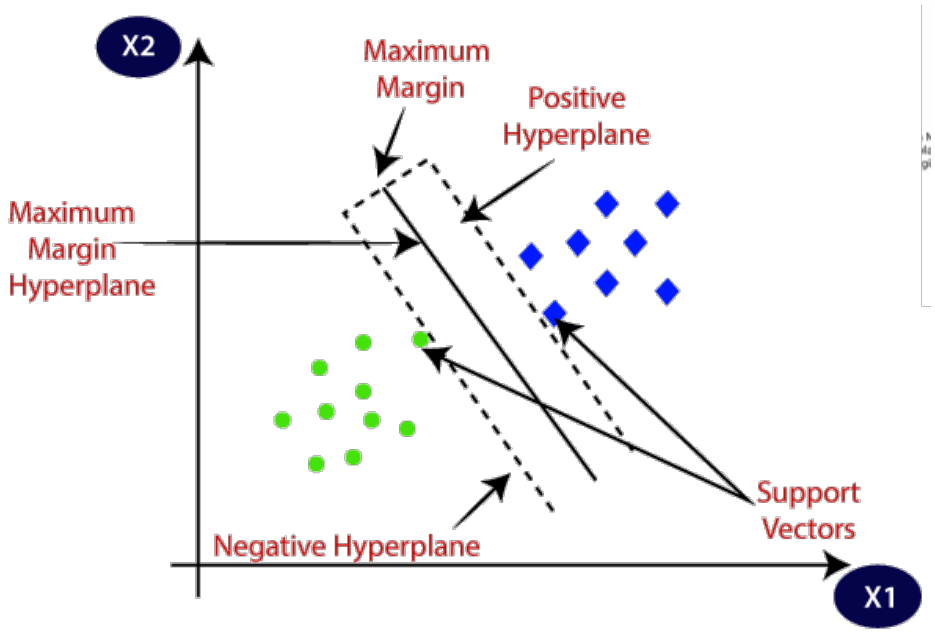


La implementación de scikit-learn soporta clasificación multiclase usando los enfoques OVR y OVO.

Introducción

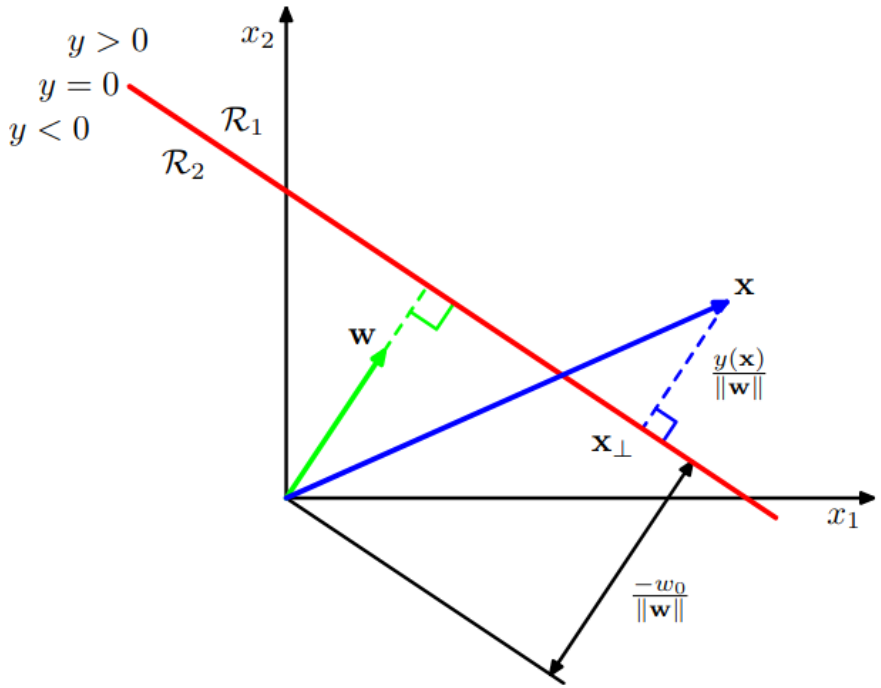
El algoritmo SVM busca los elementos de cada clase que son más similares a los de la otra clase. Estos son los *vectores de soporte*, sobre estos vectores el algoritmo busca encontrar el mejor hiperplano que los separa.

La distancia de los vectores de soporte a la frontera de decisión es el *margen*.



El método

Sea $w \in \mathbb{R}^D$ y $w_0 \in \mathbb{R}$, consideremos el hiperplano $y(x) = w^T \cdot x - t = 0$, el cual divide al espacio de datos en dos regiones.

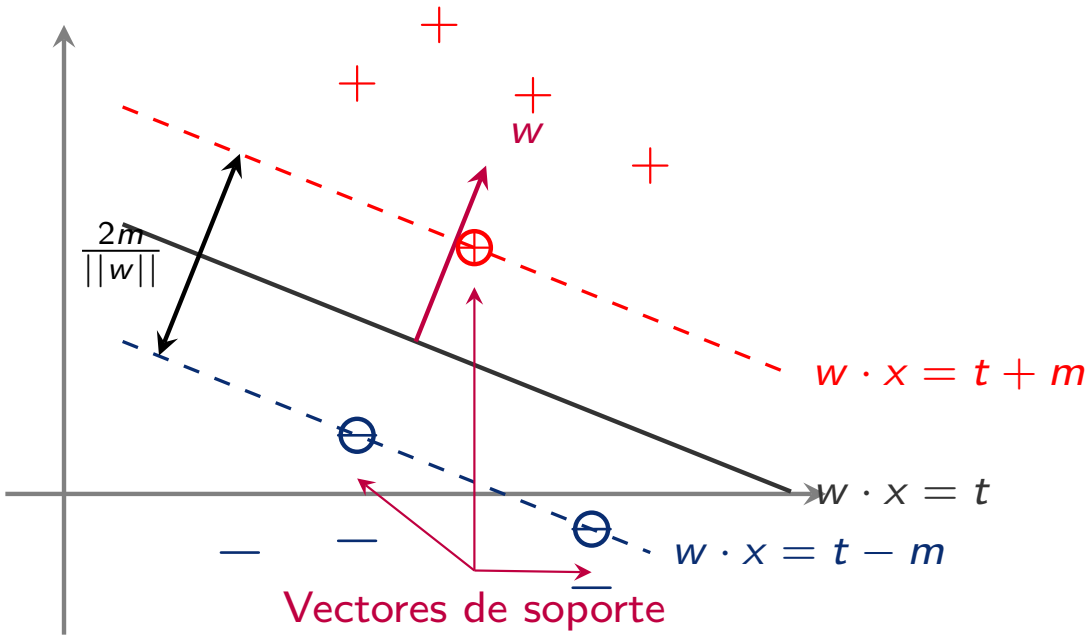


Buscamos determinar la FD usando la forma normal de la recta (vector normal y distancia al origen).

Support Vector Machine

SVM de margen duro

Analicemos el caso con datos linealmente separables. La frontera de decisión (FD) está definida por $g(x) = w^T \cdot x - t = 0$.
Queremos encontrar una FD que *maximice el margen*. Es decir, queremos minimizar $||w||$.



Support Vector Machine

Planteamiento del problema

Nuestro objetivo es encontrar w y t que minimizan:

$$\frac{1}{2}||w||^2$$

Sujeto a las N restricciones:

$$y_i(\langle w, x_i \rangle - t) \geq 1, \text{ donde } 1 \leq i \leq N$$

Multiplicadores de Lagrange

El método de los multiplicadores de Lagrange se utilizada para encontrar los máximos y mínimos locales de una función sujeta a alguna restricción.

Para optimizar una función $f(x)$ sujeta a una restricción $g(x) = 0$, se introduce una nueva variable λ (el multiplicador de Lagrange).

Se define la función Lagrangiana como:

$$\mathcal{L}(x, \lambda) = f(x) - \lambda g(x)$$

Los puntos críticos se encuentran resolviendo el sistema de ecuaciones

$$\frac{\partial}{\partial x}\mathcal{L} = 0$$
$$\frac{\partial}{\partial \lambda}\mathcal{L} = 0$$

Support Vector Machine

Usamos el método de los multiplicadores de Lagrange, con $\lambda = (\alpha_1, ..., \alpha_N)$. Definimos

$$\begin{aligned}\mathcal{L}_p(w, t, \alpha_1, \ldots, \alpha_N) &= \frac{1}{2}||w||^2 - \sum_{i=1}^{N} \alpha_i (y_i(\langle w, x_i \rangle - t) - 1) \\ &= \frac{1}{2}||w||^2 - \sum_{i=1}^{N} \alpha_i y_i \langle w, x_i \rangle + \sum_{i=1}^{N} \alpha_i y_i t + \sum_{i=1}^{N} \alpha_i \\ &= \frac{1}{2}\langle w, w \rangle - \langle w, \sum_{i=1}^{N} \alpha_i y_i x_i \rangle + t \left(\sum_{i=1}^{N} \alpha_i y_i \right) + \sum_{i=1}^{N} \alpha_i.\end{aligned}$$

Para t: $\partial_t \mathcal{L}_P = 0$ implica

$$\sum_{i=1}^{N} \alpha_i y_i = 0$$

Support Vector Machine

Usamos el método de los multiplicadores de Lagrange, con $\lambda = (\alpha_1, ..., \alpha_N)$. Definimos

$$\begin{aligned}\mathcal{L}_p(w, t, \alpha_1, \ldots, \alpha_N) &= \frac{1}{2}||w||^2 - \sum_{i=1}^{N} \alpha_i (y_i(\langle w, x_i \rangle - t) - 1) \\ &= \frac{1}{2}||w||^2 - \sum_{i=1}^{N} \alpha_i y_i \langle w, x_i \rangle + \sum_{i=1}^{N} \alpha_i y_i t + \sum_{i=1}^{N} \alpha_i \\ &= \frac{1}{2}\langle w, w \rangle - \langle w, \sum_{i=1}^{N} \alpha_i y_i x_i \rangle + t \left(\sum_{i=1}^{N} \alpha_i y_i \right) + \sum_{i=1}^{N} \alpha_i.\end{aligned}$$

Para w: $\partial_w \mathcal{L}_p = 0$ implica

$$w = \sum_{i=1}^{N} \alpha_i y_i x_i$$

Support Vector Machine

El problema y la dualidad

El problema original con restricciones equivale a un problema de optimización **min-max** sin restricciones sobre el Lagrangiano:

$$\min_{w,t} \max_{\alpha_i \geq 0} \mathcal{L}_p(w, t, \alpha_1, \ldots, \alpha_N)$$

Dado que la función y las restricciones son convexas, el teorema de dualidad fuerte nos permite invertir el orden de los operadores para obtener el problema dual:

$$\max_{\alpha_i \geq 0} \min_{w,t} \mathcal{L}_p(w, t, \alpha_1, \ldots, \alpha_N)$$

Support Vector Machine

Reinsertando estas expresiones en $\mathcal{L}_P$:

$$\begin{aligned}\mathcal{L}_D(\alpha_1, \ldots, \alpha_N) &= -\frac{1}{2}\langle \sum_{i=1}^{N} \alpha_i y_i x_i, \sum_{i=1}^{N} \alpha_i y_i x_i \rangle + \sum_{i=1}^{N} \alpha_i \\ &= -\frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle + \sum_{i=1}^{N} \alpha_i\end{aligned}$$

La solución ya sólo dependerá de los productos puntos entre datos x_i y de los coeficientes α_i. Esto es el problema dual.

Support Vector Machine

El problema de optimización dual consiste en encontrar $\alpha_1^*, \ldots, \alpha_N^*$ que maximizan la función:

$$-\frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N}\alpha_i\alpha_j y_i y_j \langle x_i, x_j\rangle + \sum_{i=1}^{N}\alpha_i$$

Sujeto a las restricciones:

$$\alpha_i > 0, \text{ para } 1 \leq i \leq N$$

$$\sum_{i=1}^{N}\alpha_i y_i = 0$$

Support Vector Machine: Solución computacional

El problema dual es un problema de *Programación Cuadrática* (QP) que nos garantiza encontrar un único máximo global.

En la práctica, ¿cómo calculan las computadoras estos α_i?

- Resolver el sistema directamente es ineficiente; el tiempo de cómputo escala de forma cuadrática con la cantidad de datos.
- Bibliotecas como `scikit-learn` (a través de `libsvm`) utilizan métodos iterativos para evadir este costo algorítmico.
- El método estándar es el algoritmo *Sequential Minimal Optimization* (SMO), que optimiza los coeficientes iterativamente tomando pares de α_i que son fáciles de resolver.

Support Vector Machine

Reconstrucción de la regla de decisión

Una vez obtenidos los coeficientes óptimos $\alpha_1^*, \ldots, \alpha_N^*$:

1. Reconstrucción del vector de pesos w^*:

$$w^* = \sum_{i=1}^{N} \alpha_i^* y_i x_i$$

2. Cálculo del término de desplazamiento t^*: Tomando cualquier vector de soporte (donde $\alpha_k^* > 0$), se cumple que $y_k(\langle w^*, x_k \rangle - t^*) = 1$, por lo que:

$$t^* = \langle w^*, x_k \rangle - y_k$$

3. Función final de clasificación para un nuevo dato x:

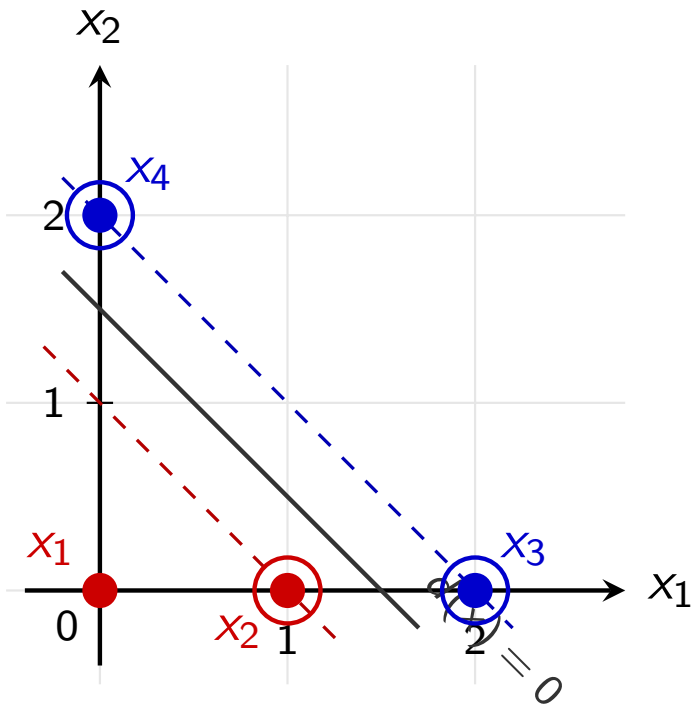
$$f(x) = \text{sign}\left(\sum_{i \in \text{SV}} \alpha_i^* y_i \langle x_i, x \rangle - t^* \right)$$

Support Vector Machine

Ejemplo

Encuentra w y t óptimo para este problema de clasificación:

$$x_1 = (0,0),\ x_2 = (1,0),\ \text{clase } +1$$
$$x_3 = (2,0),\ x_4 = (0,2),\ \text{clase } -1$$



Support Vector Machine

Ejemplo: Productos escalares

Calculamos los términos $y_i y_j \langle x_i, x_j \rangle$ para la suma doble:

$$\langle x_1, x_j \rangle = 0 \quad \forall j \implies \text{Términos con } \alpha_1 \text{ se anulan en la parte cuadrática}$$

$$y_2 y_2 \langle x_2, x_2 \rangle = (+1)(+1)(1) = 1$$

$$y_2 y_3 \langle x_2, x_3 \rangle = (+1)(-1)(2) = -2$$

$$y_2 y_4 \langle x_2, x_4 \rangle = (+1)(-1)(0) = 0$$

$$y_3 y_3 \langle x_3, x_3 \rangle = (-1)(-1)(4) = 4$$

$$y_4 y_4 \langle x_4, x_4 \rangle = (-1)(-1)(4) = 4$$

Support Vector Machine

Sustituyendo los productos escalares en la doble sumatoria de $\mathcal{L}_D$:

$$\sum_{i=1}^{4}\sum_{j=1}^{4}\alpha_i\alpha_j y_i y_j\langle x_i, x_j\rangle = \alpha_2^2 - 4\alpha_2\alpha_3 + 4\alpha_3^2 + 4\alpha_4^2$$

Sustituyendo esto en la función objetivo dual:

$$\mathcal{L}_D = (\alpha_1 + \alpha_2 + \alpha_3 + \alpha_4) - \frac{1}{2}(\alpha_2^2 - 4\alpha_2\alpha_3 + 4\alpha_3^2 + 4\alpha_4^2)$$

Acompañado de la restricción de igualdad:

$$\sum_{i=1}^{4}\alpha_i y_i = \alpha_1 + \alpha_2 - \alpha_3 - \alpha_4 = 0 \implies \alpha_1 = \alpha_3 + \alpha_4 - \alpha_2$$

Support Vector Machine

Ejemplo: Resolución del sistema

Sustituyendo $\alpha_1 = \alpha_3 + \alpha_4 - \alpha_2$ en $\mathcal{L}_D$:

$$\mathcal{L}_D(\alpha_2, \alpha_3, \alpha_4) = 2\alpha_3 + 2\alpha_4 - \frac{1}{2}\alpha_2^2 + 2\alpha_2\alpha_3 - 2\alpha_3^2 - 2\alpha_4^2$$

Maximizando mediante derivadas parciales:

$$\frac{\partial \mathcal{L}_D}{\partial \alpha_4} = 2 - 4\alpha_4 = 0 \implies \alpha_4 = 1/4$$
$$\frac{\partial \mathcal{L}_D}{\partial \alpha_2} = -\alpha_2 + 2\alpha_3 = 0 \implies \alpha_2 = 2\alpha_3$$

Evaluando en la frontera $\alpha_1 = 0$ para garantizar la factibilidad ($\alpha_1 \geq 0$):

$$\alpha_1 = \alpha_3 + 1/4 - 2\alpha_3 = 0 \implies \alpha_3 = 1/4 \quad \text{y} \quad \alpha_2 = 1/2$$

Ajustando la solución óptima del sistema dual:

$$\alpha_1 = 0, \quad \alpha_2 = 1, \quad \alpha_3 = 3/4, \quad \alpha_4 = 1/4$$

Support Vector Machine

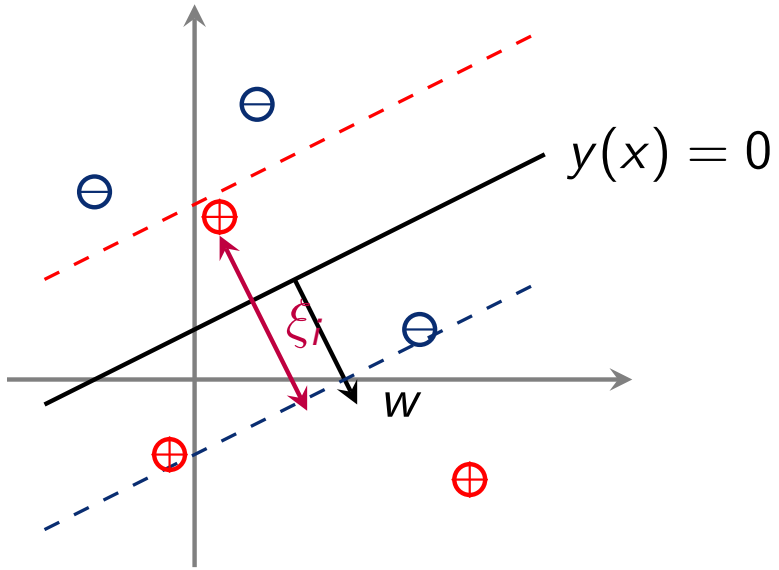
Ejemplo

Aplicando: $w = \sum_{i=1}^{N} \alpha_i y_i x_i$, finalmente obtenemos

$$w = \begin{bmatrix} -1/2 \\ -1/2 \end{bmatrix}, \quad w_0 = 3/4 \quad \text{y} \quad d(x) = 3 - 2x_1 - 2x_2 = 0$$

Support Vector Machine de margen suave

La SVM anterior no funciona con datos no separables. Introducimos variables de holgura ξ_i para cada dato de entrada, lo que les permite a algunos de ellos estar dentro del margen, o incluso del lado equivocado de la frontera de decisión.



Support Vector Machine

SVM de margen suave

Queremos encontrar w, t y ξ_i que minimicen:

$$\frac{1}{2}||w||^2 + C\sum_{i=1}^{N}\xi_i$$

sujeto a las siguientes restricciones para $1 \leq i \leq N$:

$$y_i(\langle w, x_i\rangle - t) \geq 1 - \xi_i \quad \text{y} \quad \xi_i \geq 0$$

C es un hiperparámetro que balancea la maximización del margen contra la minimización de las variables de holgura:

- Un valor alto de C significa que los errores de margen son altamente costosos.
- Un valor pequeño de C permite más errores con tal de hacer más grande el margen global.

Support Vector Machine

SVM de margen suave

Buscamos soluciones mediante el nuevo Lagrangiano:

$$\mathcal{L} = \frac{1}{2}||w||^2 + C\sum_{i=1}^{N}\xi_i - \sum_{i=1}^{N}\alpha_i(y_i(\langle w, x_i\rangle - t) - (1-\xi_i)) - \sum_{i=1}^{N}\beta_i\xi_i$$

Agrupando los términos de holgura obtenemos:

$$= \mathcal{L}(w, t, \alpha_i) + \sum_{i=1}^{N}(C - \alpha_i - \beta_i)\xi_i$$

Support Vector Machine

SVM de margen suave

Al pasar al problema dual tenemos:

$$\alpha_1^*, \ldots, \alpha_N^* = \text{argmax}_{\alpha_1,\ldots,\alpha_N} - \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N}\alpha_i\alpha_j y_i y_j \langle x_i, x_j\rangle + \sum_{i=1}^{N}\alpha_i$$

Sujeto a las restricciones:

$$0 \leq \alpha_i \leq C, \text{ para } 1 \leq i \leq N$$

$$\sum_{i=1}^{N}\alpha_i y_i = 0$$

Support Vector Machine

El hiperparámetro C

Valor alto de C:

- El modelo se enfoca en clasificar correctamente todos los ejemplos del conjunto de entrenamiento, incluso si eso significa tener un margen más pequeño.
- Penaliza más fuertemente los errores de clasificación.
- Puede resultar en un modelo que se ajuste demasiado a los datos de entrenamiento (overfitting).

Valor bajo de C:

- Permite más errores de clasificación en el conjunto de entrenamiento para lograr un margen de separación más amplio.
- Resulta en un modelo más generalizado y menos propenso a overfitting.

Table of Contents

1 Introducción

2 Support Vector Machine
- SVM de margen duro
- SVM de margen suave

3 Kernel Trick

Table of Contents

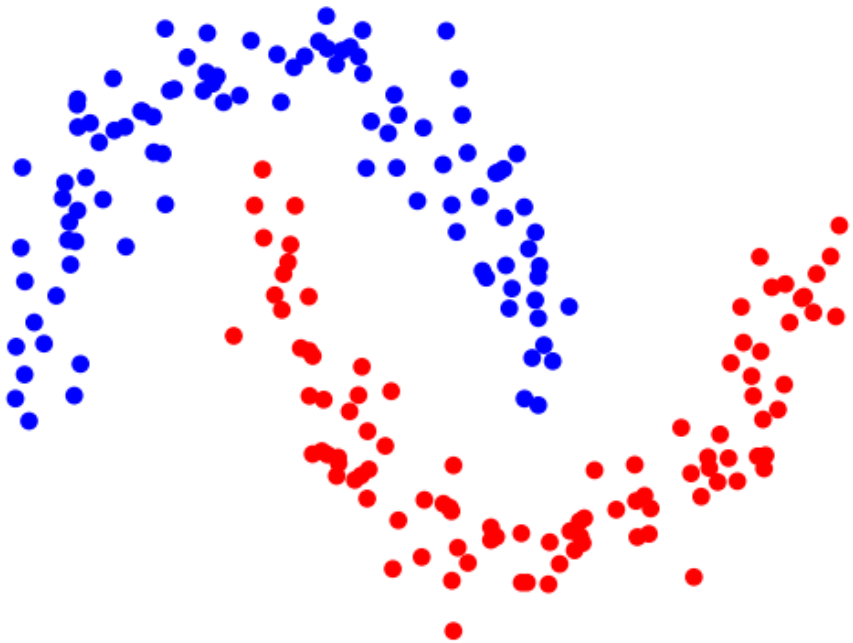
1 Introducción

2 Support Vector Machine
- SVM de margen suave

3 Kernel Trick

El truco del Kernel

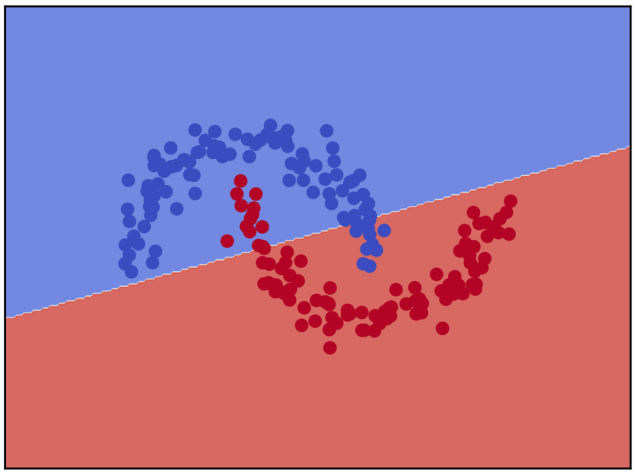
El truco del kernel consiste en sustituir el producto interior usual en la formulación del algoritmo SVM por un nueva función, llamada kernel. Esto nos permite encontrar fronteras de decisión que no necesariamente son hiperplanos.



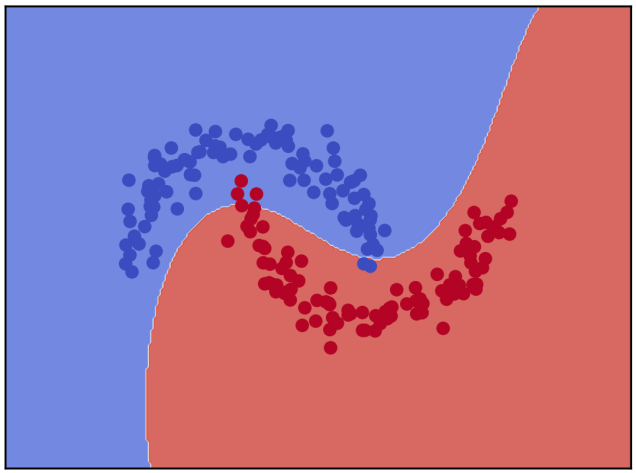
El truco del Kernel

El truco del kernel consiste en sustituir el producto interior usual en la formulación del algoritmo SVM por un nueva función, llamada kernel. Esto nos permite encontrar fronteras de decisión que no necesariamente son hiperplanos.

Kernel Lineal



Kernel Polinomial

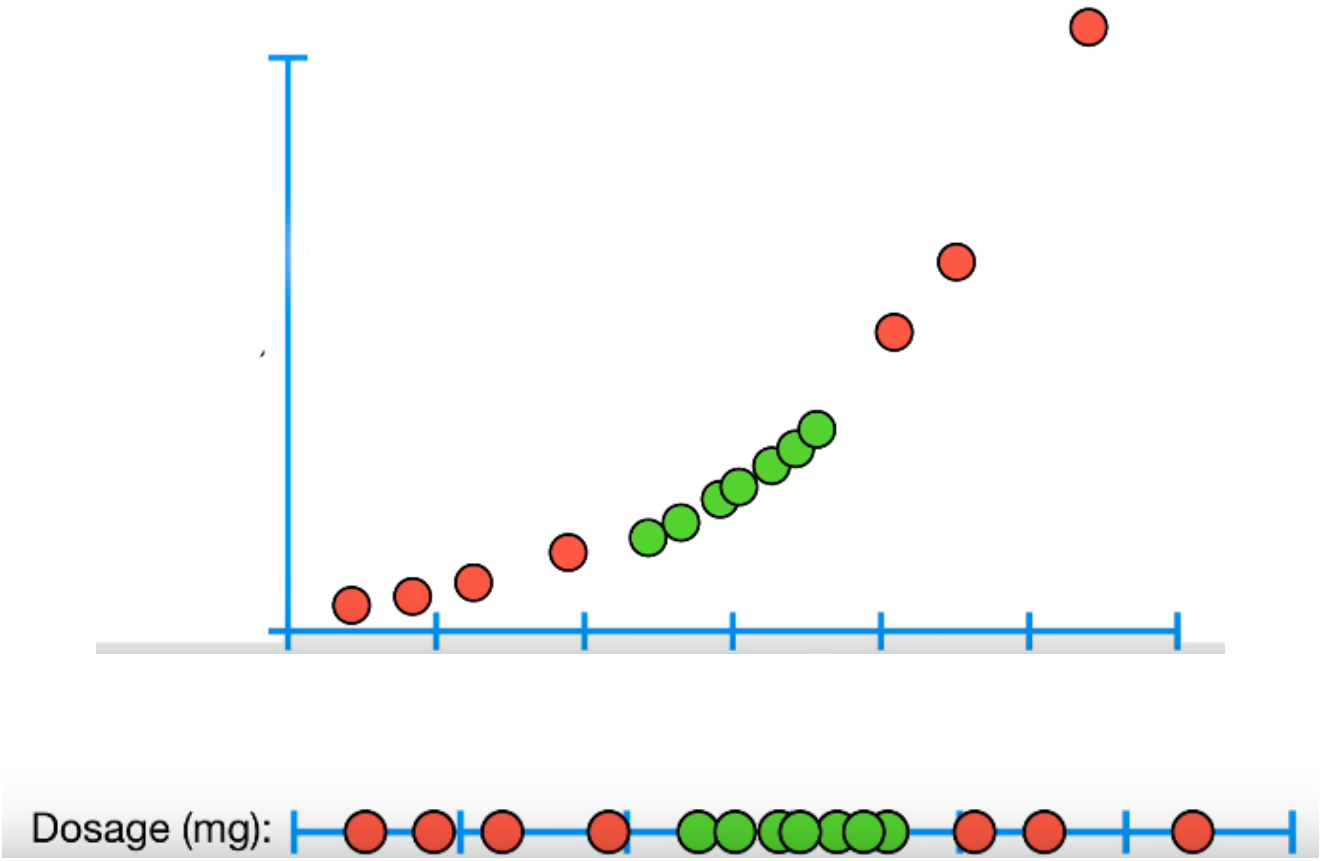


El truco del Kernel

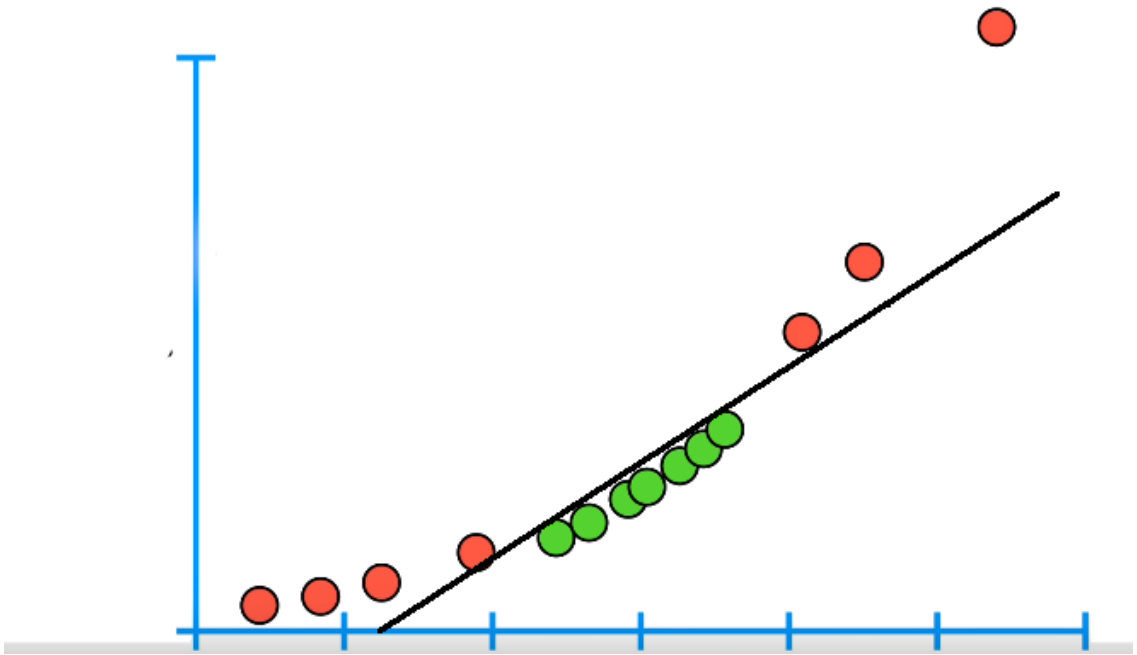
Supongamos que queremos determinar qué dosis de un medicamento es la adecuada para tratar alguna condición. Coloreamos en verde las dosis que lograron una mejoría y en rojo las que no. ¿Cómo logramos una separación lineal de los datos?

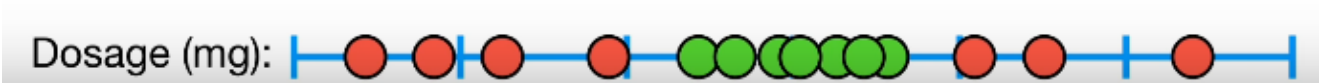


El truco del Kernel

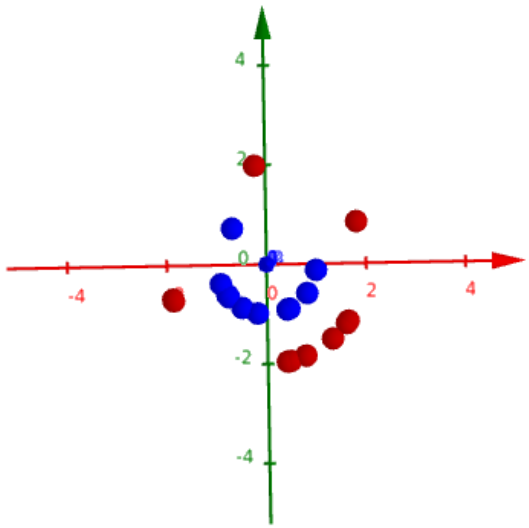


El truco del Kernel





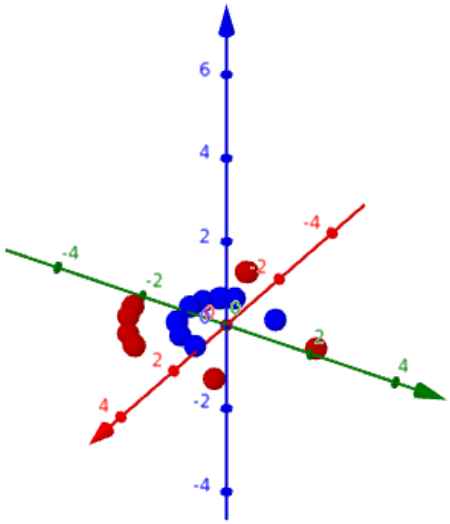
Un ejemplo con "kernel polinomial"



Queremos separar con *linealmente* estos datos.

https://www.geogebra.org/m/xawkavxe

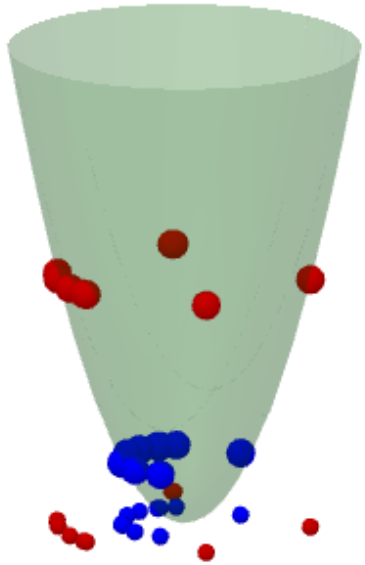
Un ejemplo con “kernel polinomial”



Consideramos los datos en un espacio de dimensión superior.

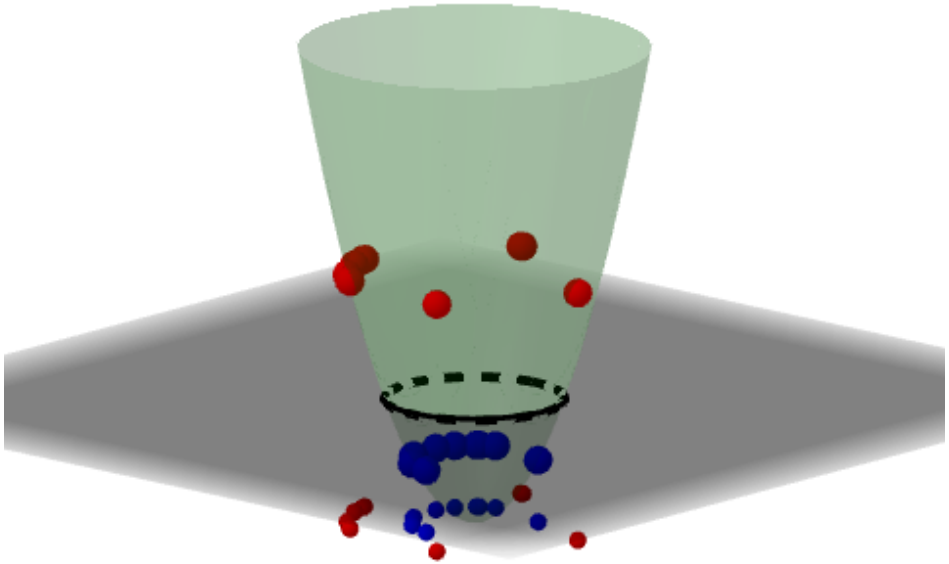
https://www.geogebra.org/m/xawkavxe

Un ejemplo con “kernel polinomial”



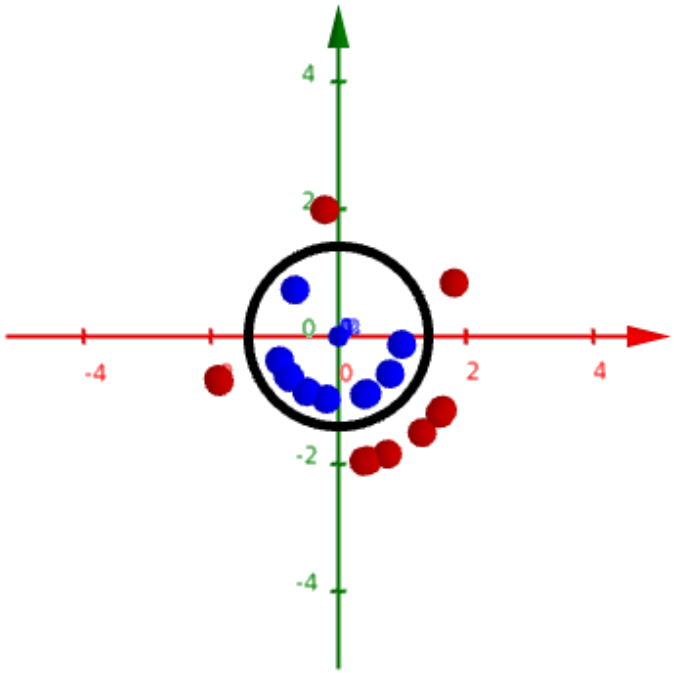
https://www.geogebra.org/m/xawkavxe

Un ejemplo con “kernel polinomial”



https://www.geogebra.org/m/xawkavxe

Un ejemplo con “kernel polinomial”



https://www.geogebra.org/m/xawkavxe

El ejemplo más sencillo: kernel polinomial de grado 2

Para puntos en $\mathbb{R}^2$, reemplazamos las 2 features originales por 4 features nuevas por medio de la función $\phi : \mathbb{R}^2 \to \mathbb{R}^4$ dada por

$$\phi(x_1, x_2) = (x_1^2, x_1 x_2, x_1 x_2, x_2^2)$$

El ejemplo más sencillo: kernel polinomial de grado 2

Para puntos en $\mathbb{R}^2$, reemplazamos las 2 features originales por 4 features nuevas por medio de la función $\phi : \mathbb{R}^2 \to \mathbb{R}^4$ dada por

$$\phi(x_1, x_2) = (x_1^2, x_1x_2, x_1x_2, x_2^2)$$

El producto interior en estas 4 features es

$$\begin{aligned}
\langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle &= \langle (x_1^2, x_1x_2, x_1x_2, x_2^2), (y_1^2, y_1y_2, y_1y_2, y_2^2) \rangle \\
&= x_1^2y_1^2 + x_1x_2y_1y_2 + x_1x_2y_1y_2 + x_2^2y_2^2 \\
&= x_1y_1x_1y_1 + x_1y_1x_2y_2 + x_1y_1x_2y_2 + x_2y_2x_2y_2 \\
&= (x_1y_1 + x_2y_2)^2 = \langle (x_1, x_2), (y_1y_2) \rangle^2 = \langle \mathbf{x}, \mathbf{y} \rangle^2
\end{aligned}$$

El ejemplo más sencillo: kernel polinomial de grado 2

Para puntos en $\mathbb{R}^2$, reemplazamos las 2 features originales por 4 features nuevas por medio de la función $\phi : \mathbb{R}^2 \to \mathbb{R}^4$ dada por

$$\phi(x_1, x_2) = (x_1^2, x_1x_2, x_1x_2, x_2^2)$$

El producto interior en estas 4 features es

$$\begin{aligned}\langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle &= \langle (x_1^2, x_1x_2, x_1x_2, x_2^2), (y_1^2, y_1y_2, y_1y_2, y_2^2) \rangle \\ &= x_1^2y_1^2 + x_1x_2y_1y_2 + x_1x_2y_1y_2 + x_2^2y_2^2 \\ &= x_1y_1x_1y_1 + x_1y_1x_2y_2 + x_1y_1x_2y_2 + x_2y_2x_2y_2 \\ &= (x_1y_1 + x_2y_2)^2 = \langle (x_1, x_2), (y_1y_2) \rangle^2 = \langle \mathbf{x}, \mathbf{y} \rangle^2\end{aligned}$$

Nos podemos ahorrar la definición de la función ϕ si definimos el kernel

$$\kappa(\mathbf{x}, \mathbf{y}) = \langle \mathbf{x}, \mathbf{y} \rangle^2$$

El siguiente ejemplo más sencillo

Para puntos en $\mathbb{R}^2$, reemplazamos las 2 features originales por 7 features nuevas por medio de la función $\phi : \mathbb{R}^2 \to \mathbb{R}^7$ dada por

$$\phi(x_1, x_2) = (x_1^2, x_1x_2, x_1x_2, x_2^2, \sqrt{2r}x_1, \sqrt{2r}x_2, r)$$

El siguiente ejemplo más sencillo

Para puntos en $\mathbb{R}^2$, reemplazamos las 2 features originales por 7 features nuevas por medio de la función $\phi : \mathbb{R}^2 \to \mathbb{R}^7$ dada por

$$\phi(x_1, x_2) = (x_1^2, x_1x_2, x_1x_2, x_2^2, \sqrt{2r}x_1, \sqrt{2r}x_2, r)$$

El producto interior en estas 7 features es

$$\begin{aligned}\langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle &= \ldots \\ &= (x_1y_1 + x_2y_2 + r)^2 \\ &= (\langle (x_1, x_2), (y_1y_2) \rangle + r)^2 = (\langle \mathbf{x}, \mathbf{y} \rangle + r)^2\end{aligned}$$

El siguiente ejemplo más sencillo

Para puntos en $\mathbb{R}^2$, reemplazamos las 2 features originales por 7 features nuevas por medio de la función $\phi : \mathbb{R}^2 \to \mathbb{R}^7$ dada por

$$\phi(x_1, x_2) = (x_1^2, x_1x_2, x_1x_2, x_2^2, \sqrt{2r}x_1, \sqrt{2r}x_2, r)$$

El producto interior en estas 7 features es

$$\begin{aligned}\langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle &= \ldots \\ &= (x_1y_1 + x_2y_2 + r)^2 \\ &= (\langle (x_1, x_2), (y_1y_2) \rangle + r)^2 = (\langle \mathbf{x}, \mathbf{y} \rangle + r)^2\end{aligned}$$

Nos podemos ahorrar la definición de la función ϕ si definimos el kernel

$$\kappa(\mathbf{x}, \mathbf{y}) = (\langle \mathbf{x}, \mathbf{y} \rangle + r)^2$$

Tipos de Kernel

Los cuatro principales kernels son:

Lineal	$\kappa(\mathbf{x}, \mathbf{y}) = \langle \mathbf{x}, \mathbf{y} \rangle$
Polinomial	$\kappa(\mathbf{x}, \mathbf{y}) = (\langle \mathbf{x}, \mathbf{y} \rangle + r)^p,\ r \geq 0$
Gaussiano (Radial Basis Function)	$\kappa(\mathbf{x}, \mathbf{y}) = \mathrm{e}^{-\gamma \|\mathbf{x}-\mathbf{y}\|^2}$
Sigmoide	$\kappa(\mathbf{x}, \mathbf{y}) = \tanh\left(\langle \mathbf{x}, \mathbf{y} \rangle + r\right)$

Algunos criterios para escoger el kernel

- Primero veamos si los datos son linealmente separables.
- El kernel lineal suele ser bueno cuando hay muchas features. Además, es el más rápido.
- El kernel polinomial es una generalización del kernel lineal.
- El kernel RBF es un buen kernel por defecto. Suele usarse cuando no hay información adicional sobre los datos.
- El kernel sigmoide equivale a usar una red neuronal de tipo *perceptron de dos capas*.
- Se pueden probar los demás kernels usando *grid seach* y *validación cruzada*.

Algunos criterios para escoger el kernel

- Primero veamos si los datos son linealmente separables.
- El kernel lineal suele ser bueno cuando hay muchas features. Además, es el más rápido.
- El kernel polinomial es una generalización del kernel lineal.
- El kernel RBF es un buen kernel por defecto. Suele usarse cuando no hay información adicional sobre los datos.
- El kernel sigmoide equivale a usar una red neuronal de tipo *perceptron de dos capas*.
- Se pueden probar los demás kernels usando *grid seach* y *validación cruzada*.

Cawley, G. C., & Talbot, N. L. (2010). On over-fitting in model selection and subsequent selection bias in performance evaluation. The Journal of Machine Learning Research, 11, 2079-2107.